

Hadaf Android Build — Troubleshooting Log & Patterns to Avoid

Session date: 2026-07-19 • Turning the Hadaf repo into a working, installable Android APK via EAS Build

This document records every issue hit while turning this repo into a working Android APK, how each was diagnosed and fixed, and — more importantly — the **recurring patterns** behind them so the same mistakes aren't repeated as the project grows. Every fix here was traced through the actual EAS cloud build logs, not guesswork.

Starting point

The repo is a pnpm monorepo. The mobile app lives at `artifacts/mobile` (Expo SDK 54, React Native 0.81.5, Expo Router, a custom native module at `artifacts/mobile/modules/hadaf-native`). There is **no android/ folder checked in** — it's generated on demand via `expo prebuild`, or built entirely in the cloud via EAS. Opening the repo root in Android Studio directly does not and cannot work; this is a Node/Expo project, not a native Android project.

Issue 1 — Kotlin compile errors in the custom native module FIXED

Symptom: `BUILD FAILED` at `:hadaf-native:compileReleaseKotlin`:

```
e: AppBlockerModule.kt:45:44 Return type mismatch: expected 'Any?', actual 'Unit'.
```

Root cause: Expo Modules' `Function("name") { ... }` Kotlin DSL requires the lambda to resolve to `Any?`. Five functions ended their body with an `if` block with no `else` — which Kotlin types as `Unit`, not `Any?`.

Fix: Added an explicit trailing `Unit` after each such block.

Pattern to watch for: Any Expo Modules `Function { ... }` block used purely for side effects should end with an explicit `Unit`, or have an explicit return type. Don't rely on Kotlin's implicit statement typing — it silently changes based on whether an `else` branch exists.

Issue 2 — EAS project linked to the wrong account FIXED

Symptom:

```
You don't have the required permissions to perform this operation.  
Entity not authorized: AppEntity[...] (viewer = ..., action = READ)
```

Root cause: `app.config.js` had a hardcoded `extra.eas.projectId` pointing at an EAS project owned by whoever originally built this app — not the new Expo account logged in on this machine.

Fix: Removed the stale `projectId`, ran `eas init --non-interactive --force` to create a fresh project under the correct account, then manually added the new `projectId` / `owner` back into `app.config.js` (dynamic configs can't be auto-patched by the CLI).

Pattern to watch for: Forking/cloning an Expo project with a baked-in `projectId`? Check `eas whoami` matches the project's actual owner before building — this looks like a permissions bug but is really "wrong project."

Issue 3 — Android resource merge conflict FIXED

Symptom:

```
Execution failed for task ':app:mergeReleaseJavaResource'.  
2 files found with path 'META-INF/versions/9/OSGI-INF/MANIFEST.MF' from inputs:  
- com.squareup.okhttp3:logging-interceptor:5.4.0  
- org.jspecify:jspecify:1.0.0
```

Root cause: Two transitive dependencies both bundle a file at the same `META-INF` path.

Fix: Added the `expo-build-properties` config plugin with a `packagingOptions.exclude` entry — declarative, since there's no checked-in `android/` folder to hand-edit.

Pattern to watch for: Don't eject and hand-edit generated Gradle files — they're rebuilt on every `prebuild` /EAS build and your edit will silently vanish. Use `expo-build-properties` for this class of fix.

Issue 4 — Installing Expo-ecosystem packages with the wrong tool

FIXED — MOST IMPORTANT PATTERN

Symptom: After `pnpm add expo-build-properties`, a later build failed deep inside `node_modules`:

```
Execution failed for task ':expo-dev-menu:compileDebugKotlin'.  
e: DevMenuModule.kt:14:34 Unresolved reference 'OptimizedRecord'.
```

`expo doctor` revealed the real story:

package	expected	found
---------	----------	-------

expo-build-properties	~1.0.10	57.0.6
expo-dev-client	~6.0.21	57.0.5

Root cause: `pnpm add / npm install` grabs the npm `latest` tag — which for Expo-ecosystem packages can be many major versions ahead of what the installed SDK (54) actually supports.

Fix: `npx expo install --fix` realigned every package to its SDK-54-compatible version.

★ The single most important pattern from this session:

Never `pnpm add / npm install` a package starting with `expo-` or `@expo/`. Always use `npx expo install <package>` — it looks up the compatible version for your installed SDK. Run `npx expo-doctor` before any build when unsure; it catches this in seconds instead of a 20-minute failed cloud build.

Issue 5 — App installs but crashes instantly on open FIXED

Symptom: APK installs and opens, then immediately closes / shows nothing. No build error — this is a runtime crash.

Root cause: `app/_layout.tsx` wraps the app in `<ClerkProvider publishableKey={process.env.EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY!}>`. The `!` is only a TypeScript assertion — meaningless at runtime. The EAS build had **zero environment variables configured**, so the key was `undefined`. Clerk's SDK throws synchronously when initialized with no key, and that throw happens *inside* `ClerkProvider` — which is a *parent* of the app's own `ErrorBoundary`, not a child. Nothing caught it; the JS thread crashed before any UI rendered.

Fix:

1. Created a Clerk account/app, got a publishable key.
2. Registered it via `eas env:create` across all three build environments (development/preview/production).
3. Pulled it locally too (`eas env:pull`) for the dev-client workflow, and added `.env*` to `.gitignore` first.

Pattern to watch for: Local dev scripts can hide missing configuration — this repo's `dev` script manually exported the key from Replit's secret store, so it "just worked" locally while never being wired into the actual EAS build pipeline. Any `EXPO_PUBLIC_*` value your code reads must be explicitly registered as an **EAS Environment Variable**; it won't carry over automatically from wherever it worked before. Also worth a follow-up: a missing required env var should never be a silent crash — consider validating it up front with a clear on-screen message instead of letting the provider throw unguarded.

Issue 6 — Android OS install friction (expected behavior, not bugs)

- **Play Protect blocks the install** → "More details" → "Install anyway" in the warning dialog.
- **Usage Access / Accessibility toggles greyed out** → Android 13+ anti-malware protection for sideloaded apps. Fix: **Settings** → **Apps** → **Hadaf** → **:** menu → **"Allow restricted settings"**.

Pattern to watch for: These happen on *every* fresh install of any internal-distribution APK, on every device. Not a broken build — worth a one-time note in tester onboarding.

Issue 7 — Dev-client "failed to connect" for a remote tester FIXED

Symptom: A second tester, not on the same network as the machine running Metro, opens the dev-client app and gets:

```
Error loading app
failed to connect to /192.168.0.103 (port 8081) from /192.168.1.100 (port 50682) after 10000ms
```

Root cause: `npx expo start --dev-client` (no `--tunnel`) serves Metro over plain LAN — the dev-client app connects directly to the host machine's local IP. That IP is only reachable from devices on the **same Wi-Fi network** as the host. A tester on a different network is on a different subnet entirely and the connection times out — basic networking, not an app or build bug.

Fix: Each person testing against a live Metro server runs `npx expo start --dev-client` **from their own machine**, with their own phone on the same Wi-Fi as that machine. No single shared dev server works for testers on different networks unless using `--tunnel` (routes through ngrok, works from anywhere, adds latency).

Note: `--tunnel` itself failed on first try in this environment with an `@expo/ngrok` binary-resolution error (`ERR_INVALID_ARG_TYPE`) — not debugged further since LAN mode was available instead. Worth revisiting if a tester's phone and laptop are ever on different networks from *each other* too.

Pattern to watch for: Dev-client live debugging is inherently local to whoever's machine runs `expo start`. If multiple people need to test simultaneously against live Metro/HMR, each needs their own `expo start` running locally — plan for that instead of assuming one dev server can serve a remote collaborator. A static `preview` / `production` APK remains the most reliable way to hand a working build to someone remote, with zero networking dependency.

Debugging setup now in place

A **development-client** build is now available instead of only opaque preview/production APKs:

- Built via the existing `development` profile in `eas.json` (debug build, faster — no minification/lint-vital).

- Run `npx expo start --dev-client` from `artifacts/mobile`, connect the dev-client app to `exp://<machine-LAN-IP>:8081`.
- JS errors now show as an in-app **red-screen overlay with a full stack trace**, and all console output streams live to the terminal.

This is the tool to use for the "further issue" mentioned at the end of this session — reproduce it with the dev client connected, and the real error/stack trace is visible immediately.

Quick reference — commands used this session

```
# One-time environment setup
npm install -g pnpm eas-cli
pnpm install
eas login

# Installing Expo-ecosystem packages correctly
npx expo install <package-name>      # NOT pnpm add / npm install
npx expo-doctor                       # sanity check before every build

# EAS environment variables
eas env:create --name <NAME> --value <VALUE> \
  --environment development --environment preview --environment production \
  --visibility plaintext --non-interactive
eas env:pull --environment development

# Builds
eas build --platform android --profile development --non-interactive  # dev-client
eas build --platform android --profile preview --non-interactive       # distributable APK

# Live debugging
npx expo start --dev-client      # connect device to exp://<lan-ip>:8081

# Fetching full logs for a failed cloud build
eas build:view <build-id> --json      # get data.logFiles[0]
curl -s --compressed "<logFiles[0] URL>" # Brotli NDJSON; --compressed decodes it
```

Status at end of session

- ✓ Preview APK builds and installs successfully.
- ✓ App opens without crashing (Clerk key wired up).
- ✓ Dev-client build + live Metro debugging set up for ongoing troubleshooting.
- ✓ Project handed off to a second collaborator; documented the "each tester needs their own local `expo start`" networking requirement (Issue 7) after they hit it directly.
- ⚠ Still open: the "further issue" mentioned after the app first opened successfully — not yet described/diagnosed. Next step: reproduce with a dev client connected to a local Metro server to capture the exact error.